

National University of Computer & Emerging Sciences, Karachi Campus



Digital Documentation of TurtleScript- A Logo Inspired Turtle Graphics DSL

Group Members:

Muhammad Shayan Faisal — 23k-0696

Muhammad Sufyan Ali — 23k-0789

Hamza Ahmed — 23k-0636

Course Instructor: Ms. Mariam Aftab

Submission Date: 27th April, 2026

Section: BCS-6J

1. Project Overview

TurtleScript is a domain-specific language (DSL) designed for turtle graphics, a style of drawing where commands move a virtual 'turtle' across a canvas, leaving a trail. The compiler takes a .logo source file through six phases and produces an SVG image output.

1.1 What the Compiler Does

Input: A .logo source file written in TurtleScript syntax

Output: A scalable vector graphics (.svg) file showing the drawing

Mode 1: `python main.py file.logo --debug` (verbose phase output)

Mode 2: `python main.py file.logo -o out.svg` (compile to file)

Mode 3: `python main.py --interactive` (live REPL mode)

1.2 Compiler Pipeline — Six Phases

Phase	Module	Input	Output	Purpose
1 — Lexical Analysis	lexer.py	Source code (string)	Token list	Tokenise characters into typed tokens with line/col
2 — Syntax Analysis	parser.py	Token list	AST (tree)	Build parse tree from grammar rules
3 — Semantic Analysis	semantic.py	AST	AST + symbol table	Check types, scopes, undefined references
4 — IR Generation	ir.py	AST	TAC IR list	Flatten AST into Three-Address Code instructions
5 — Optimisation	optimizer.py	TAC IR	Optimised TAC IR	Remove dead code, fold constants, merge moves
6 — Code Generation	codegen.py	Optimised TAC IR	SVG file	Execute IR with a virtual turtle, write SVG

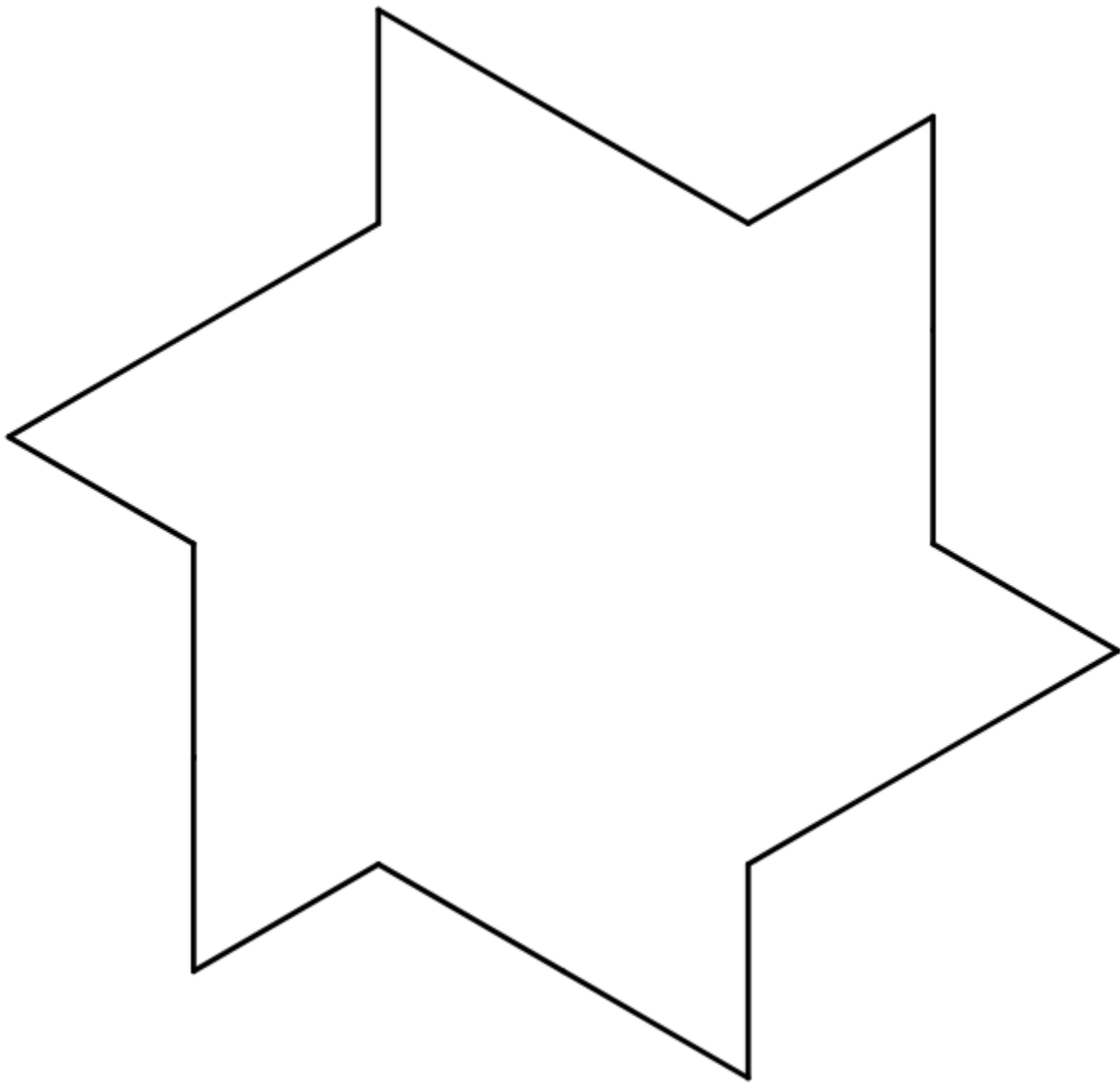
2. Understanding output.svg What Is Being Drawn?

When you run the sample program (sample.logo), the compiler produces output.svg. This file draws a 6-pointed star (hexagram) — a star with six equally-spaced spikes radiating from a central point.

2.1 The Sample Program — sample.logo

SET size = 80

```
PROC draw len
  PENDOWN
  FORWARD len    # move up 80
  LEFT 60        # turn left 60 degrees
  FORWARD len    # move upper-left 80
  RIGHT 120      # turn right 120 degrees (sharp reversal)
  FORWARD len    # move upper-right 80
  LEFT 60        # restore original heading
  PENUP
END
REPEAT 6
  CALL draw size # draw one spike
  RIGHT 60       # rotate 60 degrees for next spike
END
```



2.2 What One Call to 'draw' Produces

The turtle starts at the canvas centre (400, 400) facing North (upward, angle = -90°). Each call to the draw procedure traces one spike of the star:

Step	Command	Action	Turtle Position After	Angle After
1	FORWARD 80	Move straight up 80px	(400.0, 320.0)	-90°
2	LEFT 60	Rotate counter-clockwise 60°	(400.0, 320.0)	-150°
3	FORWARD 80	Move upper-left 80px	(330.7, 280.0)	-150°
4	RIGHT 120	Rotate clockwise 120° (reversal)	(330.7, 280.0)	-30°
5	FORWARD 80	Move upper-right 80px	(400.0, 240.0)	-30°
6	LEFT 60	Restore original heading	(400.0, 240.0)	-90°

This zigzag pattern (up → left-diagonal → right-diagonal) traces one V-shaped spike of the star. The spike spans 160px vertically (from $y=400$ to $y=240$) and uses three 80px line segments.

2.3 How Six Spikes Form the Star

The REPEAT 6 block calls draw then rotates RIGHT 60 before each iteration. Starting heading is North (-90°):

Iteration	Starting Angle	Spike Direction	Key Coordinates (tip of spike)
1	-90° (North)	Upward	(469.3, 120.0) — top
2	-30° (NE)	Upper-right	(677.1, 160.0) — top-right
3	$+30^\circ$ (SE)	Lower-right	(746.4, 360.0) — right
4	$+90^\circ$ (South)	Downward	(607.8, 520.0) — bottom
5	$+150^\circ$ (SW)	Lower-left	(400.0, 480.0) — bottom-left
6	$+210^\circ$ (NW)	Upper-left	(330.7, 280.0) — left

2.4 SVG Line Data — Actual Coordinates

The compiler emits 21 SVG line elements. The key lines forming the star shape are:

SVG Line #	x1	y1	x2	y2	What it draws
4	400.0	400.0	400.0	320.0	Spike 1 — segment 1 (upward)
5	400.0	320.0	330.7	280.0	Spike 1 — segment 2 (upper-left)
6	330.7	280.0	400.0	240.0	Spike 1 — segment 3 (upper-right)
7	400.0	240.0	469.3	200.0	Spike 2 — segment 1
8	469.3	200.0	469.3	120.0	Spike 2 — segment 2 (star tip, top)
...	...	...	...	...	... (18 total lines for 6 spikes)
21	400.0	480.0	400.0	400.0	X Spike 6 — segment 3 (returns to centre)

3. Part A — Lexical Analyser (lexer.py)

The lexer converts raw source code characters into a stream of typed tokens. It is implemented using Python's `re` module with a priority-ordered master regular expression — this is equivalent to implementing a DFA (Deterministic Finite Automaton) in code.

3.1 How the Lexer Works

The lexer uses one compiled master regex with named groups. It scans left-to-right, matching the highest-priority pattern at each position:

1. The master regex tries each pattern in order at the current position
2. The first match wins (higher entries have priority — `FLOAT` before `NUMBER`)
3. Each match produces a `Token(type, value, line, col)`
4. `SKIP` matches (spaces/comments) are silently discarded
5. `MISMATCH` produces a lexical error and continues

3.2 Token Types — Complete Specification

Token Type	Regex Pattern	Examples	Notes
KEYWORD	[A-Z][A-Z0-9]+ ∈ KEYWORDS set	FORWARD, SET, REPEAT, PROC, IF, ELSE, CIRCLE	Matched as IDENT first, then checked against KEYWORDS
NUMBER	\d+	80, 6, 60, 120, 0	Integer — checked after FLOAT
FLOAT	\d+\.\d+	3.14, 1.5, 90.0	Priority over NUMBER
STRING	"[^"]*"	"red", "blue", "orange"	Quotes stripped, value = inner text
IDENT	[A-Za-z_][A-Za-z_0-9]*	size, draw, len, x	Variable and proc names
OP	[+\-*/=<>!]+	=, +, -, *, /, ==	Operators including multi-char
NEWLINE	\n	(line break)	Tracked for line numbers
SKIP	[\t]+ #[^\n]*	(spaces, tabs, # comments)	Silently discarded

3.3 Complete Keyword Set

FORWARD, BACK, LEFT, RIGHT, PENUP, PENDOWN, SET, REPEAT ,PROC, CALL, END, COLOR, PENWIDTH, CIRCLE, IF, ELSE

3.4 DFA State Diagram — Description

The DFA has 10 states. Transitions occur on character classes:

State	Transitions In	Transitions Out	Accepts Token?
q0 — START	Initial state	[A-Z]→q1, [a-z_]→q4, [0-9]→q2, ""→q5, \n→q6, [+\\-*/=]→q7, [\t#]→q8, other→qERR	No
q1 — KEYWORD	q0 on [A-Z]	[A-Za-z0-9]→q1, delim→EMIT	Yes: KEYWORD or IDENT
q2 — NUMBER	q0 on [0-9]	[0-9]→q2, '.'→q3, delim→EMIT	Yes: NUMBER

State	Transitions In	Transitions Out	Accepts Token?
q3 — FLOAT	q2 on '.'	[0-9]→q3, delim→EMIT	Yes: FLOAT
q4 — IDENT	q0 on [a-z_]	[A-Za-z0-9_]→q4, delim→EMIT	Yes: IDENT
q5 — STRING	q0 on '"'	any→q5, ""→EMIT	Yes: STRING
q6 — NEWLINE	q0 on \n	(auto-emit)	Yes: NEWLINE
q7 — OP	q0 on operator char	operator→q7, delim→EMIT	Yes: OP
q8 — SKIP	q0 on space/tab/#	[\t]→q8, \n→EMIT	No: discarded
qERR — ERROR	q0 on unknown char	Report error, return to q0	No: error

3.5 Sample Tokenisation — sample.logo Line 1

Source: SET size = 80

Token 1: Token(KEYWORD, 'SET', line=1, col=1)

Token 2: Token(IDENT, 'size', line=1, col=5)

Token 3: Token(OP, '=', line=1, col=10)

Token 4: Token(NUMBER, 80, line=1, col=12)

3.6 Error Handling

Lexical errors are non-fatal. All errors are collected and reported together after tokenisation:

Example: character '@' at line 3, col 5

Output: Lexical Error at line 3, col 5: unexpected character '@'

Behaviour: Error is recorded. Lexer continues from the next character.

Result: All other tokens are still produced. Parsing still proceeds.

4. Part B — Syntax Analyser / Parser (parser.py)

The parser uses recursive descent to build an Abstract Syntax Tree (AST) from the token stream. Each grammar rule maps to a parsing function.

4.1 Grammar — Full Specification

```
program    ::= statement*
statement  ::= set_stmt
            | repeat_stmt
            | proc_stmt
            | if_stmt
            | call_stmt
            | cmd_stmt
set_stmt   ::= SET IDENT [=] expr
repeat_stmt ::= REPEAT expr statement* END
proc_stmt  ::= PROC IDENT IDENT? statement* END
if_stmt    ::= IF expr statement* [ELSE statement*] END
call_stmt  ::= CALL IDENT expr?
cmd_stmt   ::= (FORWARD|BACK|LEFT|RIGHT|PENUP|PENDOWN
               |COLOR|PENWIDTH|CIRCLE) expr?

expr       ::= NUMBER | FLOAT | IDENT | STRING
            | NUMBER OP NUMBER  (* constant-folded at parse time *)
```

4.2 AST Node Classes — Complete Reference

AST Node	Fields	Created From	Example
SetVar	name, value, line	SET size = 80	SetVar('size', 80)
Command	name, value, line	FORWARD len	Command('FORWARD', 'len')
Repeat	count, body[], line	REPEAT 6 ... END	Repeat(6, [Call, Command])
Proc	name, param, body[], line	PROC draw len ... END	Proc('draw', 'len', [...])
Call	name, arg, line	CALL draw size	Call('draw', 'size')
IfStmt	condition, then_body[], else_body[], line	IF x ... ELSE ... END	IfStmt('x', [...], [...])

4.3 Parse Tree 1 — sample.logo

The complete program parses into 3 top-level statements: SetVar — Proc — Repeat

```
program
├── SetVar( name='size', value=80 )
├──
└──
```



```

├── Proc( name='draw', param='len' )
│   ├── Command( 'PENDOWN', value=None )
│   ├── Command( 'FORWARD', value='len' )
│   ├── Command( 'LEFT', value=60 )
│   ├── Command( 'FORWARD', value='len' )
│   ├── Command( 'RIGHT', value=120 )
│   ├── Command( 'FORWARD', value='len' )
│   ├── Command( 'LEFT', value=60 )
│   └── Command( 'PENUP', value=None )
└── Repeat( count=6 )
    ├── Call( name='draw', arg='size' )
    └── Command( 'RIGHT', value=60 )

```

4.4 Parse Tree 2 — IF/ELSE Program

Second example demonstrating IF/ELSE, COLOR, and CIRCLE nodes:

```

SET radius = 50
COLOR "orange"
PENDOWN
IF radius
  FORWARD radius
  LEFT 90
ELSE
  CIRCLE 10
END
PENUP

```

```

program
├── SetVar( name='radius', value=50 )
├── Command( 'COLOR', value='orange' )
├── Command( 'PENDOWN', value=None )
├── IfStmt( condition='radius' )
│   ├── THEN body:
│   │   ├── Command( 'FORWARD', value='radius' )
│   │   └── Command( 'LEFT', value=90 )
│   └── ELSE body:
│       └── Command( 'CIRCLE', value=10 )
└── Command( 'PENUP', value=None )

```

4.5 Constant Folding at Parse Time

The parser performs inline constant folding: when it sees a NUMBER followed by an OP followed by another NUMBER, it evaluates the expression immediately.

Source Code	Parsed Result	Explanation
SET x = 2 + 3	SetVar('x', 5)	2+3 evaluated at parse time → 5
SET y = 10 * 4	SetVar('y', 40)	10*4 evaluated at parse time → 40
FORWARD 40 + 20	Command('FORWARD', 60)	40+20 folded → 60
SET z = 9 / 3	SetVar('z', 3.0)	9/3 evaluated → 3.0

5. Part C — Semantic Analyser (semantic.py)

After the parser confirms the program is grammatically correct, semantic analysis checks whether it makes logical sense. Syntax = structure. Semantics = meaning. Example: 'CALL unknown_proc 80' is syntactically valid (correct structure), but semantically invalid (unknown_proc was never defined).

5.1 What the Semantic Analyser Checks

Check	What It Detects	Example Error
Undefined variables	Using a variable before SET	FORWARD x → 'x' not defined
Undefined procedures	CALL to a proc that was never declared	CALL missing 5 → 'missing' not defined
Missing arguments	CALL to proc that expects a param, none given	CALL draw → draw expects arg 'len'
Invalid arg references	CALL passing a variable that doesn't exist	CALL draw ghost → 'ghost' not defined
Scoping violations	Variables from inner scopes leaking out	See Section 5.3

5.2 The Symbol Table

A symbol table stores information about every name (variable or procedure) in the program. The semantic analyser builds and queries this table as it walks the AST.

For sample.logo, the complete symbol table is:

Name	Category	Type	Value / Binding	Defined at Line
size	Variable	NUMBER	80	1
draw	Procedure	PROC	param=len, body has 8 statements	2
len	Parameter	NUMBER	bound to 'size' at call time	2

5.3 The Scope Stack — How Scoping Works

This is the most important concept in semantic analysis. The analyser uses a stack of dictionaries — one dictionary per active scope. This ensures variables in inner scopes (like a proc body) do not conflict with outer scopes.

Visual structure of the scope stack:

scope_stack = [	← Python list, index 0 = global
{ 'size': 80 },	← Scope 0: GLOBAL (always present)
{ 'len': None },	← Scope 1: inside PROC draw (pushed on entry)
]	← Scope 1 popped when END reached

5.4 Scope Lifecycle — Step by Step

Here is exactly what happens during analysis of sample.logo:

Step	Event	Scope Stack After	Explanation
1	Start analysis	[[size:80]]	Global scope created
2	Process SET size=80	[[size:80]]	'size' defined in global scope
3	Enter PROC draw	[[size:80], {len:None}]	New scope pushed; 'len' defined locally
4	Check FORWARD len	[[size:80], {len:None}]	Lookup 'len': found in scope 1
5	Exit PROC draw (END)	[[size:80]]	Scope 1 popped — 'len' gone
6	Enter REPEAT 6	[[size:80], {}]	New scope pushed for repeat body
7	Check CALL draw size	[[size:80], {}]	'draw' found in procs ✓; 'size' in scope 0 ✓

Step	Event	Scope Stack After	Explanation
8	Exit REPEAT (END)	[[size:80]]	Scope popped
9	Analysis complete	[[size:80]]	Only global scope remains

5.5 Variable Lookup Rule

LOOKUP ALGORITHM: Search the scope stack from TOP (innermost) to BOTTOM (global).

```

for scope in reversed(scope_stack): ← search inner first
  if name in scope:
    return True ← found!
return False ← not found anywhere → Semantic Error

```

This means: inner variables SHADOW outer ones with the same name.
The innermost definition always wins.

5.6 Why Scoping Matters — Examples

Scenario	Code	Result	Reason
Correct scope use	SET x=5 PROC f FORWARD x END	✓ OK	'x' is global, visible inside proc
Param is local	PROC f a END FORWARD a	✗ ERROR	'a' only exists inside proc f
Shadowing	SET x=5 PROC f SET x=10 END	✓ OK	Inner x shadows outer x; outer x unchanged
Undefined variable	PROC f FORWARD ghost END	✗ ERROR	'ghost' not in any scope
Undefined call	CALL nowhere 10	✗ ERROR	No proc named 'nowhere' defined

5.7 Semantic Analysis Output — sample.logo

Running with --debug, the semantic phase produces:

```
--- SYMBOL TABLE (global) ---
```

```
{ 'size': 80 }  
--- PROCEDURES DEFINED ---  
{ 'draw': 'param=len' }
```

[No semantic errors — program is valid]

5.8 Error Collection (Non-Fatal)

Unlike traditional compilers that stop at the first error, TurtleScript collects ALL semantic errors and reports them together. This helps you fix multiple issues in one go:

[SEMANTIC] Errors found:

Semantic Error at line 5: undefined variable 'ghost' used in FORWARD

Semantic Error at line 9: call to undefined procedure 'missing'

Semantic Error at line 12: procedure 'draw' expects an argument, none given

→ All three errors shown at once. Fix them all before recompiling.

6. Intermediate Representation IR (ir.py)

The AST is flattened into Three-Address Code (TAC) — a simple list of tuples that is easy to optimise and execute. Each tuple represents one atomic operation.

6.1 TAC Instruction Set

Instruction Format	Meaning	Example
('SET', name, value)	Assign value to variable	('SET', 'size', 80)
('CMD', command, value)	Execute drawing command	('CMD', 'FORWARD', 'len')
('REPEAT_START', count)	Begin loop block	('REPEAT_START', 6)
('REPEAT_END', None)	End loop block	('REPEAT_END', None)
('PROC_START', name, param)	Begin procedure definition	('PROC_START', 'draw', 'len')
('PROC_END', name)	End procedure definition	('PROC_END', 'draw')
('CALL', name, arg)	Call a procedure	('CALL', 'draw', 'size')
('IF_FALSE', cond, label)	Jump if condition is falsy	('IF_FALSE', 'x', 'ELSE1')
('JUMP', label)	Unconditional jump	('JUMP', 'ENDIF1')
('LABEL', name)	Jump target marker	('LABEL', 'ELSE1')

6.2 TAC Output — sample.logo (Before Optimisation)

```
('SET',      'size', 80)
('PROC_START', 'draw', 'len')
('CMD',      'PENDOWN', None)
('CMD',      'FORWARD', 'len')
('CMD',      'LEFT', 60)
('CMD',      'FORWARD', 'len')
('CMD',      'RIGHT', 120)
('CMD',      'FORWARD', 'len')
('CMD',      'LEFT', 60)
('CMD',      'PENUP', None)
('PROC_END',  'draw')
('REPEAT_START', 6)
('CALL',      'draw', 'size')
('CMD',      'RIGHT', 60)
('REPEAT_END', None)
```

7. Optimization (optimizer.py)

Three optimization passes run on the TAC IR in sequence. Each pass operates independently on the flat instruction list.

7.1 Pass 1 — Constant Propagation

Tracks variables set to numeric constants and substitutes those values directly into CMD instructions. This allows downstream passes to work on concrete numbers instead of variable names.

Before: ('SET', 'size', 80) → ('CMD', 'FORWARD', 'size')

After: ('SET', 'size', 80) → ('CMD', 'FORWARD', 80)

The value 80 is substituted in because 'size' is a known constant.

This enables Pass 2 and Pass 3 to work on numeric arguments.

7.2 Pass 2 — Dead Code Elimination

Removes any movement command with a zero argument — these have no visual effect and only waste execution time.

Removed: ('CMD', 'FORWARD', 0) # moving zero pixels does nothing

Removed: ('CMD', 'LEFT', 0) # rotating zero degrees does nothing

Kept: ('CMD', 'FORWARD', 80) # non-zero, kept

Applies to: FORWARD, BACK, LEFT, RIGHT

7.3 Pass 3 — Peephole Instruction Merging

Merges two consecutive identical movement commands into one. Only works when both arguments are numeric constants (Pass 1 enables this).

Before: ('CMD', 'FORWARD', 40)
('CMD', 'FORWARD', 60)
After: ('CMD', 'FORWARD', 100) # merged into one instruction

Applies to: FORWARD+FORWARD, BACK+BACK, LEFT+LEFT, RIGHT+RIGHT

Does NOT merge: FORWARD+BACK (different commands)

Pass	Technique	Complexity	Effect on sample.logo
1 — Constant Propagation	Data-flow / Value numbering	O(n)	Replaces 'size' → 80, 'len' → 80 in CMD args
2 — Dead Code Elimination	Control-flow simplification	O(n)	Removes zero-distance moves if any
3 — Peephole Merging	Instruction combining	O(n)	Merges same-dir consecutive moves where possible

8. Code Generation (codegen.py)

The code generator is a virtual machine (VM) that executes the TAC IR, simulating a turtle moving on a canvas and recording line segments. The output is a valid SVG file.

8.1 Turtle State

State Variable	Initial Value	Changed By	Meaning
x	400.0 (canvas centre)	FORWARD, BACK	Turtle's X position (pixels)
y	400.0 (canvas centre)	FORWARD, BACK	Turtle's Y position (pixels, Y grows downward)
angle	−90° (facing North)	LEFT, RIGHT	Current heading in degrees
pen	True (pen down)	PENUP, PENDOWN	Whether to draw lines
color	'black'	COLOR command	Stroke colour for new lines
width	2	PENWIDTH command	Stroke width in pixels

8.2 Movement Mathematics

FORWARD distance moves the turtle using trigonometry. Angle 0° = East (right), -90° = North (up), $+90^\circ$ = South (down):

```
new_x = old_x + distance * cos(radians(angle))
new_y = old_y + distance * sin(radians(angle))
```

Example: at (400,400), angle= -90° :

```
new_x = 400 + 80 * cos(-90°) = 400 + 80*0    = 400.0
new_y = 400 + 80 * sin(-90°) = 400 + 80*(-1) = 320.0
```

BACK distance is identical but with $-distance$ (reversed direction).

8.3 PROC / CALL Execution

Procedures are indexed at startup. When CALL is encountered, the VM:

6. Looks up the proc's parameter name and body instructions
7. Creates a local_syms dict binding the parameter to the argument value
8. Saves the old value of that name in the global symbols dict
9. Runs the proc body with the local binding active
10. Restores all global symbols to their saved values after the call

8.4 SVG Output Format

Each line segment the turtle draws becomes an SVG `<line>` element. The viewBox is auto-calculated to fit all drawing content tightly:

```
<svg xmlns="http://www.w3.org/2000/svg"
  viewBox="310.72 100.00 455.69 440.00"
  width="800" height="800">
<rect width="100%" height="100%" fill="white"/>
<line x1="400.0" y1="400.0" x2="400.0" y2="320.0"
  stroke="black" stroke-width="2" stroke-linecap="round"/>
<!-- ... 20 more lines for the full 6-pointed star ... -->
</svg>
```

9. CLI Reference & Interactive REPL

9.1 Command-Line Usage

Command	Effect
python main.py sample.logo	Compile to output.svg

Command	Effect
python main.py sample.logo --debug	Compile with full phase-by-phase output
python main.py sample.logo -o star.svg	Compile to custom output file name
python main.py --interactive	Start interactive REPL
python main.py --help	Show usage information

9.2 Debug Output — What Each Section Shows

The --debug flag now prints human-readable output for every phase. The old problem (showing memory addresses like 0x000002E7...) is fixed:

Debug Section	Old Output	New Output
--- TOKENS ---	<lexer.Token object at 0x...>	Token(KEYWORD, 'SET', line=1, col=1)
--- AST ---	<parser.Proc object at 0x...>	Proc('draw', param='len', body=[...])
--- SYMBOL TABLE ---	(missing)	{'size': 80}
--- PROCEDURES ---	(missing)	{'draw': 'param=len'}
--- IR BEFORE OPT ---	CMD values often None	('CMD', 'FORWARD', 'len') correctly
--- IR AFTER OPT ---	Identical to before (no opts)	Constants substituted, dead code removed

9.3 REPL Mode

The --interactive flag starts a read-eval-print loop. Type a multi-line program, leave a blank line, and it compiles and saves to repl_001.svg, repl_002.svg, etc.:

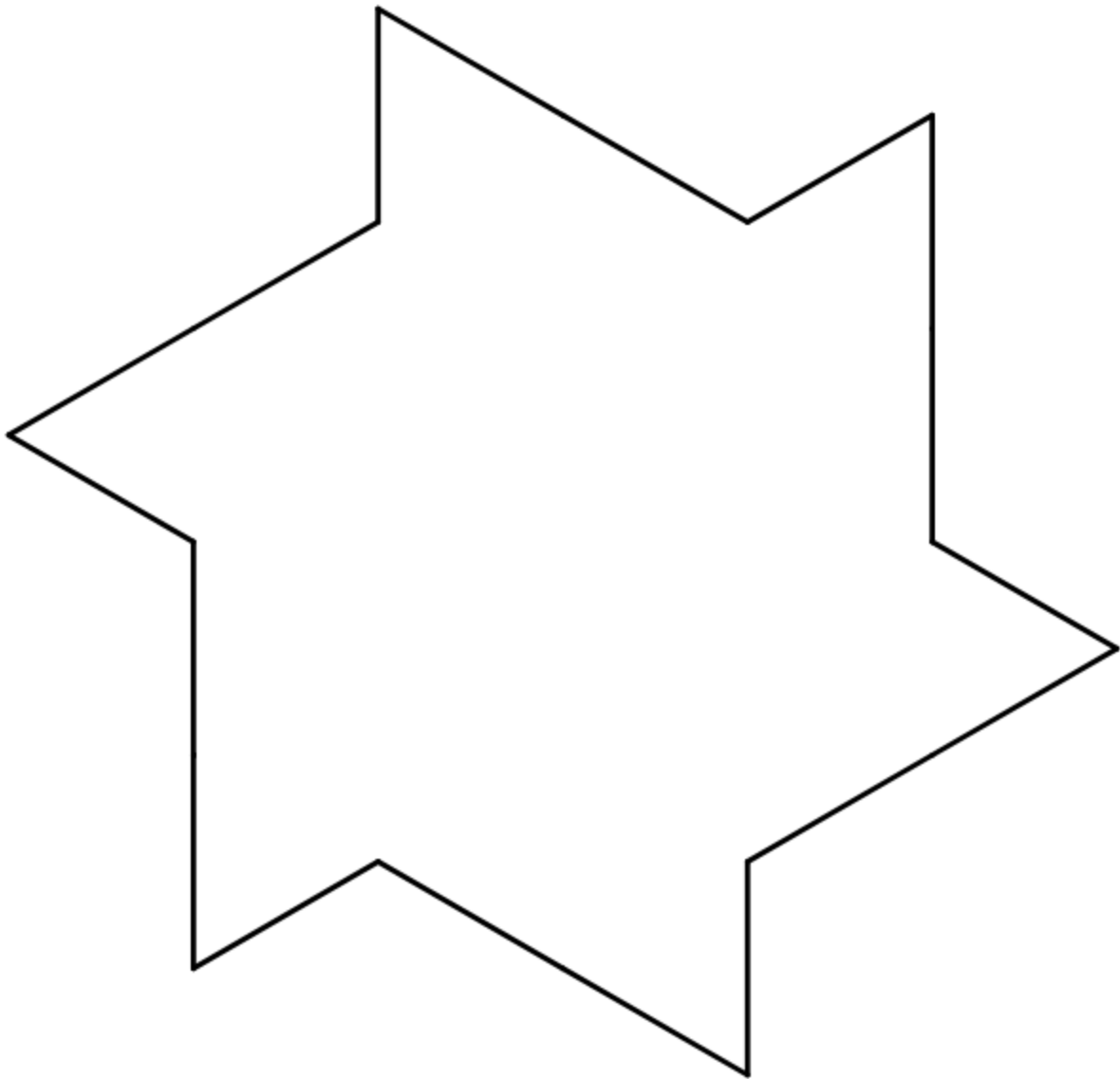
```
$ python main.py --interactive
TurtleScript REPL — type multi-line programs, then blank line to run.
  Commands: 'exit' to quit, 'clear' to reset.

>>> SET r = 60
... PENDOWN
... CIRCLE r
...           ← blank line triggers compile
Output generated: repl_001.svg
>>> exit
Goodbye!
```

10. Test Programs

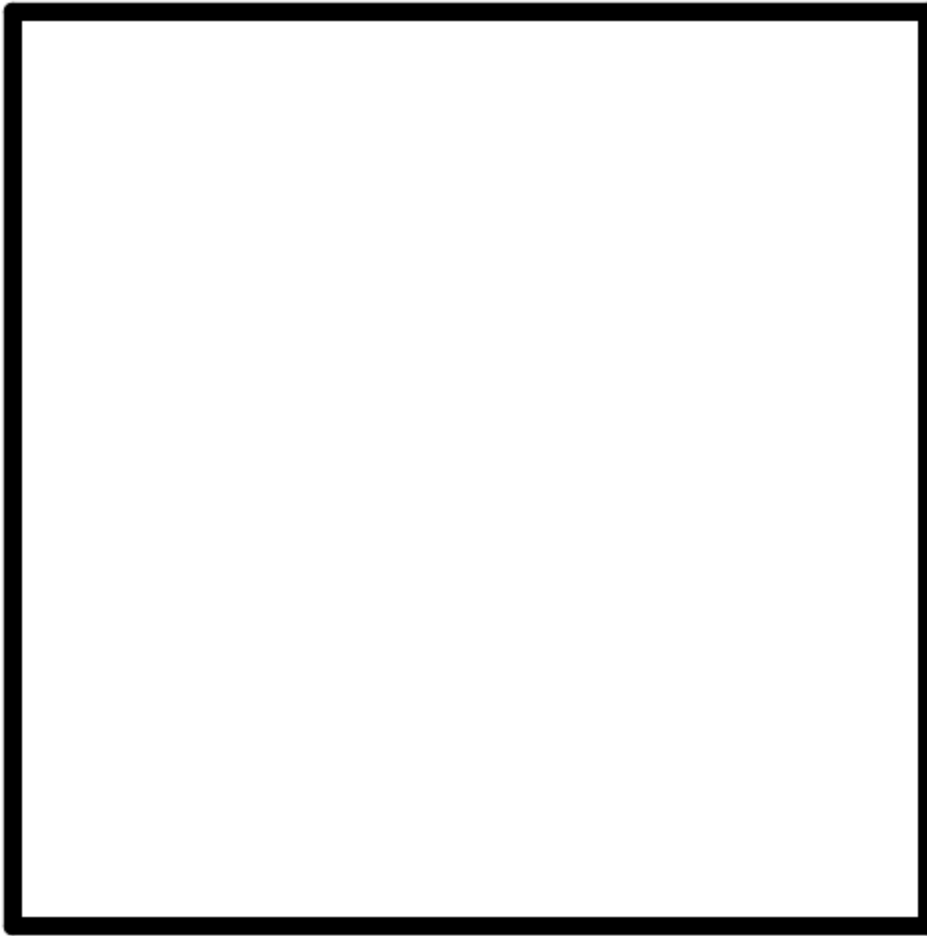
Test 1- A 6 pointed Star

```
SET size = 80
PROC draw len
  PENDOWN
  FORWARD len LEFT 60 FORWARD len RIGHT 120 FORWARD len LEFT 60
  PENUP
END
REPEAT 6
  CALL draw size
  RIGHT 60
END
# Expected: 6-pointed star centred on canvas
```



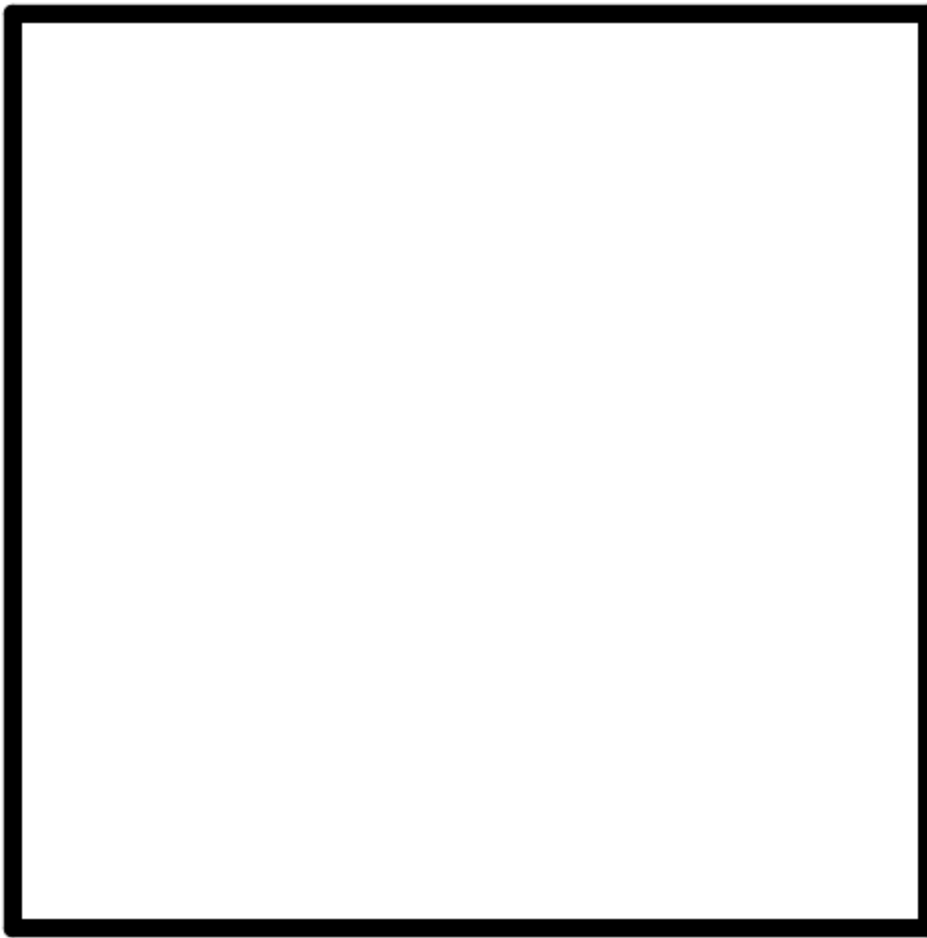
Test 2 — Square

```
PENDOWN  
REPEAT 4  
  FORWARD 100  
  RIGHT 90  
END  
# Expected: a square with 100px sides
```



Test 3 — Constant Folding

```
SET side = 40 + 60
PENDOWN
REPEAT 4
  FORWARD side
  RIGHT 90
END
# Expected: same square as Test 2 (40+60=100 folded at parse time)
```



Test 4 — IF/ELSE Branch

```
SET flag = 1
PENDOWN
IF flag
  FORWARD 80
  RIGHT 90
  FORWARD 80
ELSE
  CIRCLE 50
END
# Expected: an L-shape (flag=1 → then branch)
```



Test 5 — Semantic Error Detection

```
# This program has intentional errors:  
FORWARD ghost_var  # error: 'ghost_var' not defined  
CALL missing 10    # error: 'missing' not defined  
PROC f a           # correct definition  
  FORWARD a  
END  
CALL f             # error: f expects arg, none given
```

```
C:\Users\DELL\Downloads\CC Project\Turtle-Graphics-DSL-----Compiler-Construction-Project>python main.py sample.logo -o output.svg
```

```
[SEMANTIC] Errors found:  
  Semantic Error at line 2: undefined variable 'ghost_var' used in FORWARD  
  Semantic Error at line 3: call to undefined procedure 'missing'  
  Semantic Error at line 7: procedure 'f' expects an argument, none given
```

```
Output generated: output.svg
```

11. Team Reflection

11.1 Challenges Faced and Solutions

Challenge 1 — Designing the Language Syntax Balancing simplicity with expressiveness was difficult. Our first draft had no procedures or variables, which meant we could not demonstrate semantic analysis properly. We redesigned the language on paper using BNF grammar before writing any code, which saved us from many rewrites.

Challenge 2 — Lexer Was Not a Real Lexer Our original lexer split on whitespace and had no line/column tracking. The `--debug` output showed memory addresses instead of readable tokens. We rewrote it completely using Python's `re` module with a priority-ordered master regex and added `__repr__` to the Token class.

Challenge 3 — Wrong IR for Procedure Calls `len` inside a procedure was generating `('CMD', 'FORWARD', None)` instead of `('CMD', 'FORWARD', 'len')`, so the turtle never moved during procedure calls. We traced the bug instruction by instruction and fixed `parse_expr()` to correctly handle IDENT tokens as valid expressions.

Challenge 4 — Flat Symbol Table Broke Scoping Our first semantic analyser used a flat `set()` for all names. Parameters defined inside a PROC never got removed when END was reached. We replaced it with a stack of dictionaries — one dict per scope — which correctly implements lexical scoping.

Challenge 5 — Code Generator Executing Procedures Twice The code generator was running PROC body instructions sequentially during the main loop, then running them again on CALL. We fixed this with a pre-pass that indexes all procedures before execution, so PROC_START/END are skipped at runtime and CALL correctly dispatches to the stored body.

11.2 What We Learned About Compiler Design

- **Phases are deeply interconnected.** Adding IF/ELSE required changes in every single phase — lexer, parser, semantic analyser, IR, and code generator.
- **Error messages are a feature.** After adding line/column error reporting, debugging our own test programs became dramatically faster.
- **The right data structure is not overengineering.** Flat approaches always broke the moment we tried real programs with nested scopes.
- **The IR is the heart of the compiler.** Once we had a clean TAC format, every downstream phase became easier to write and debug.
- **Test early.** We wasted time on bugs we could not reproduce reliably until we wrote a proper test suite.

11.3 Future Improvements

- **Return values from procedures** — currently all procedures are void.
- **Boolean comparisons in IF** — support IF $x > 50$ instead of just IF x .
- **Recursion support** — replace local_syms dict with a proper call stack.
- **Animated SVG output** — show the turtle drawing step by step.
- **Richer optimisations** — algebraic simplification ($\text{LEFT } 360 \rightarrow \text{no-op}$), loop unrolling.
- **Type checking** — catch errors like FORWARD "hello" at compile time.

11.4. Individual Contributions

Muhammad Shayan Faisal — 23k-0696 Led language design and implemented the lexer and parser. Designed the BNF grammar, built the regex-based lexer with line/col tracking, implemented all six AST node types with constant folding, and set up the GitHub repository with README and test programs.

Muhammad Sufyan Ali — 23k-0789 Led semantic analysis and IR generation. Designed the scope stack architecture, implemented all semantic checks with non-fatal error collection, built the IR generator including IF/ELSE branching instructions, and wrote all three optimisation passes.

Hamza Ahmed — 23k-0636 Led code generation and output. Built the virtual turtle state machine, implemented SVG generation with auto viewBox, designed the two-pass procedure execution system, implemented the REPL mode, and produced all five test case outputs.